

TRABALHO PRÁTICO

Detecção de Fraude em Transações Pix com MLP

Classificador com rede neural densa, normalização, regularização e comparação de funções de ativação

1. Contexto de Negócio

Você é cientista de dados em uma fintech brasileira que processa **3 milhões de transações Pix por dia**. A equipe de prevenção a fraudes identificou que o sistema baseado em regras atual detecta apenas 45% das fraudes, resultando em **R\$ 2,3 milhões de prejuízo mensal**.

Sua missão é construir um modelo de **rede neural MLP (Multi-Layer Perceptron)** que classifique transações como legítimas ou fraudulentas, aplicando técnicas de normalização, regularização e otimização vistas em aula.

2. Descrição do Dataset

O arquivo **transacoes_pix_fraude.csv** contém 10.000 transações Pix com 20 features e 1 variável target (**fraude**). A taxa de fraude é de aproximadamente 8%, simulando um cenário realista de desbalanceamento.

2.1 Dicionário de Variáveis

Variável	Tipo	Descrição
valor_transacao	Float	Valor da transação em R\$ (5 a 80.000)
hora_transacao	Int	Hora do dia (0–23). Fraudes concentram-se de madrugada
dia_semana	Int	0=Segunda ... 6=Domingo
qtd_transacoes_24h	Int	Número de transações nas últimas 24 horas
tempo_desde_ultima_transacao_min	Float	Minutos desde a última transação. Contém missing (~3%)
distancia_localizacao_km	Float	Distância em km entre a transação e localização usual. Contém missing
idade_conta_dias	Int	Dias desde a criação da conta
qtd_dispositivos_distintos	Int	Dispositivos diferentes usados na conta
score_credito	Int	Score de crédito (300–850). Contém missing (~3%)
renda_mensal_estimada	Float	Renda mensal estimada em R\$. Contém missing (~3%)
qtd_transacoes_mes	Int	Total de transações no mês
media_valor_transacao_30d	Float	Média de valor das transações nos últimos 30 dias
desvio_valor_media	Float	Desvio relativo do valor em relação à média de 30 dias
mesmo_destinatario_recente	Bin	1=destinatário já recebeu Pix recentemente

tipo_chave_pix	Cat	0=CPF, 1=Celular, 2=Email, 3=Aleatória
tentativas_senha_errada	Int	Tentativas de senha errada antes da transação
usa_vpn	Bin	1=transação realizada via VPN
transacao_internacional	Bin	1=IP de origem internacional
valor_acumulado_24h	Float	Valor total acumulado nas últimas 24h
razao_valor_renda	Float	Razão entre valor da transação e renda mensal
fraude	Bin	TARGET: 0=legítima, 1=fraude (~8% positivo)

3. Tarefas do Exercício

Execute cada etapa a seguir no Jupyter Notebook. Documente suas observações e conclusões em células Markdown entre cada bloco de código.

Etapa 1 — Análise Exploratória e Tratamento de Dados

1. Carregue o dataset `transacoes_pix_fraude.csv` e realize uma análise exploratória básica (shape, dtypes, describe, distribuição da variável target).
2. Identifique e trate os valores ausentes (missing data). Escolha uma estratégia de imputação (média, mediana ou KNN Imputer) e **justifique sua escolha**.
3. Separe X (features) e y (target). Faça o split train/validation/test na proporção 70% / 15% / 15%, com `stratify=y`.

Etapa 2 — Normalização

Aplique normalização nas features e compare o impacto. Esta etapa é fundamental para redes neurais, pois features em escalas diferentes prejudicam o gradiente descendente.

4. Treine um modelo MLP **SEM nenhuma normalização** (baseline). Registre a acurácia, F1-Score e AUC-ROC no conjunto de validação.
5. Aplique `StandardScaler` (média 0, desvio 1) e retreine o modelo. Compare as métricas.
6. Aplique `MinMaxScaler` (escala 0–1) e retreine o modelo. Compare com o `StandardScaler`.
7. Responda: qual scaler produziu melhores resultados? Por quê? (Dica: analise a distribuição das features — há outliers?)

Atenção: Faça o fit do scaler APENAS nos dados de treino e aplique transform em validação e teste. Isso evita data leakage.

Etapa 3 — Modelo Base (MLP sem Regularização)

Construa o modelo MLP base usando a normalização que teve melhor resultado na Etapa 2.

Arquitetura sugerida:

Camada	Neurônios	Ativação	Regularização
Dense 1	64	ReLU	Nenhuma (agora)
Dense 2	32	ReLU	Nenhuma (agora)
Dense 3 (saída)	1	Sigmoid	—

8. Compile com `optimizer='adam'` e `loss='binary_crossentropy'`.
9. Treine por 100 épocas com `batch_size=32`.
10. Plote as curvas de loss e accuracy (treino vs validação).
11. **Responda: há sinais de overfitting? O que você observa nas curvas?**

Etapa 4 — Regularização (Dropout + L2 + Early Stopping)

Agora aplique técnicas de regularização para combater overfitting. Teste cada técnica **individualmente** e depois **combinadas**.

12. **Modelo A — Apenas Dropout:** Adicione Dropout(0.3) após cada camada Dense. Treine e compare métricas.
13. **Modelo B — Apenas L2 (Ridge):** Adicione `kernel_regularizer=l2(0.01)` em cada camada Dense. SEM Dropout. Treine e compare.
14. **Modelo C — Apenas L1 (Lasso):** Substitua L2 por `kernel_regularizer=l1(0.01)`. SEM Dropout. Treine e compare.
15. **Modelo D — Dropout + L2 + Early Stopping:** Combine Dropout(0.3), L2(0.01) e `EarlyStopping(patience=10, restore_best_weights=True)`. Treine com `epochs=200`.
16. Monte uma tabela comparativa com: Accuracy, Precision, Recall, F1-Score e AUC-ROC de cada modelo (A, B, C, D) no conjunto de validação.

 **Importante:** Em fraude, Recall (sensibilidade) costuma ser mais importante que Accuracy. Uma fraude não detectada custa muito mais do que um falso alarme. Leve isso em conta na análise.

Etapa 5 — Comparação de Funções de Ativação

Usando a melhor combinação de regularização da Etapa 4, teste diferentes funções de ativação nas camadas ocultas:

Função	Fórmula	Característica
ReLU	$\max(0, x)$	Padrão para redes densas. Simples e eficiente. Pode sofrer de 'dying ReLU'.
LeakyReLU	$\max(0.01x, x)$	Evita dying ReLU permitindo pequeno gradiente para valores negativos.
ELU	$\alpha(e^x - 1)$ se $x < 0$	Saída média mais próxima de zero. Pode melhorar convergência.
Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	Saída entre -1 e 1. Centrada em zero. Pode sofrer vanishing gradient.
Swish	$x \cdot \text{sigmoid}(x)$	Proposta pelo Google. Suave e não-monótona. Bons resultados em redes profundas.

17. Para cada função de ativação, treine o modelo com a mesma arquitetura e regularização.
18. Monte uma tabela comparativa com F1-Score e AUC-ROC.
19. Plote as curvas de treinamento de TODAS as funções em um único gráfico (subplots).
20. **Responda: qual função convergiu mais rápido? Qual obteve melhor resultado final? Alguma sofreu vanishing gradient?**

Etapa 6 — Avaliação Final e Conclusão

21. Selecione o melhor modelo (melhor combinação de normalização + regularização + ativação).
22. Avalie no **conjunto de TESTE** (usado apenas nesta etapa!).
23. Gere e analise: Confusion Matrix, Classification Report, Curva ROC.
24. Calcule o impacto financeiro: se cada fraude não detectada custa R\$ 4.200 e cada falso alarme custa R\$ 50 (custo operacional de investigação), qual o custo total do seu modelo vs o baseline?
25. **Escreva um parágrafo de conclusão respondendo:**
 - a) A normalização fez diferença? Por quê?
 - b) Qual regularização teve maior impacto? Dropout, L1 ou L2?
 - c) A função de ativação importou muito ou pouco neste caso?
 - d) Você colocaria este modelo em produção? O que faria diferente?

4. Entregáveis

- **Jupyter Notebook (.ipynb)** com todo o código, gráficos e análises em células Markdown.
- **Tabela comparativa final** (em Markdown no notebook) com todos os modelos testados.
- **Análise de impacto financeiro** com cálculos explícitos.

5. Critérios de Avaliação

Critério	Peso	O que será avaliado
Tratamento de dados (missing + split)	15%	Estratégia e justificativa
Normalização comparativa	15%	Comparação e análise crítica
Regularização (Dropout, L1, L2, Early Stopping)	25%	Implementação e comparação
Funções de ativação	15%	Teste e interpretação
Avaliação final + impacto financeiro	15%	Métricas e análise de negócio
Clareza do código e documentação	15%	Código limpo + Markdown

6. Dicas e Imports Úteis

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_auc_score, f1_score, roc_curve)
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.regularizers import l1, l2, l1_l2
from tensorflow.keras.callbacks import EarlyStopping
import matplotlib.pyplot as plt
```